

ATLANTIC COMPUTATIONAL EXCELLENCE NETWORK

Introduction to Shell Scripting

Dr. Ross Dickson, Computational Research Consultant

What is the Shell?

It is...

- ...a program
- ...a programming language
- ...how you interact with the operating system

A first shell script

first.sh

```
# Comments are prefixed with a # mark

echo "It is now" `date`
echo
who
echo
echo "My user name is" $(whoami) "and I am in
    directory" $(pwd)
```

The output is:

```
rdickson@glooscap: ~/demo > chmod +x first.sh
```

```
rdickson@glooscap: ~/demo > ./first.sh
```

```
It is now Thu Oct 30 10:54:20 ADT 2008
```

```
skhan      pts/0      Oct 30 08:29 (140.184.24.201)
```

```
skhan      pts/3      Oct 30 08:47 (140.184.24.201)
```

```
vchevrie   pts/5      Oct 30 09:42 (happy.phys.dal.ca)
```

```
rdickson   pts/6      Oct 30 10:41 (rdickson.ucis.dal.ca)
```

```
My user name is rdickson and I am in directory
```

```
/home/rdickson/demo
```

```
rdickson@glooscap: ~/demo >
```

Interpolation

` ` backquotes and \$() dollar-parentheses
both *interpolate*:

Run the inner command and substitute the standard output into the outer command string before running it.

```
# Comments are prefixed with a # mark

echo "It is now" `date`
echo
who
echo
echo "My user name is" $(whoami) "and I am in
    directory" $(pwd)
```

A practical example:

Suppose you have
fifty jobs on the cluster,
some in r state,
some in qw state,
and some in Eqw state.

How do you delete your Eqw jobs efficiently?

qstat output

```
$ qstat
```

job-ID	prior	name	user	state	submit/start	at	queue
4320603	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:22:58	long.q
4320606	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:23:14	long.q
4320607	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:23:14	long.q
4320604	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:22:58	long.q
4320605	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:22:58	long.q
4320600	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:22:28	long.q
4319822	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:17	
4319823	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:22	
4319826	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:30	
4319828	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:35	
4319829	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:41	
4319831	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:47	
4319837	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:05	
4319839	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:11	
4319841	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:17	
4319843	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:23	

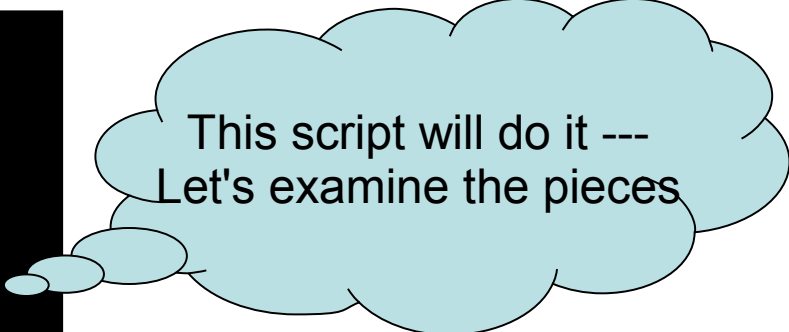
We need to 'qdel 4319837', 'qdel 4319839',

The answer

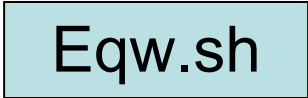
Suppose you have fifty jobs on the cluster, some in *r* state, some in *qw* state and some in *Eqw* state

How do you delete your *Eqw* jobs efficiently?

```
qstat > temp1
grep 'Eqw' temp1 > temp2
cut -c1-8 temp2 > temp3
sed 's/^/qdel /' temp3 > del.sh
source del.sh
rm temp1 temp2 temp3 del.sh
```

A light blue thought bubble with a black outline, containing text. It has three smaller circles leading to it from the left.

This script will do it ---
Let's examine the pieces

A light blue rectangular box with a black border, containing the text "Eqw.sh".

Eqw.sh

Find lines with *grep*

```
$ qstat
```

job-ID	prior	name	user	state	submit/start	at	queue
4320605	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:22:58	long.q
4320600	0.50000	qgenNettsZ	jksmith	r	05/21/2010	15:22:28	long.q
4319829	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:41	
4319831	0.50000	qpostPro	jksmith	qw	05/21/2010	14:13:47	
4319837	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:05	
4319839	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:11	
4319841	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:17	
4319843	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:23	

grep 'Eqw' temp1

4319837	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:05	
4319839	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:11	
4319841	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:17	
4319843	0.50000	qpostPro	jksmith	Eqw	05/21/2010	14:14:23	

grep: “Regular expressions”

```
grep 'Jacob' phonebook
```

```
Jacob 902 555 1234
```

```
grep -v 'Jac' phonebook  
...everything but Jac...
```

```
James 902 555 4567  
Jane 902 555 2365  
Keith 902 555 5673  
Kenny 902 555 0989  
Larry 902 555 6575
```

phonebook			
Jacob	902	555	1234
James	902	555	4567
Jane	902	555	2365
Keith	902	555	5673
Kenny	902	555	0989
Larry	902	555	6575

PATTERNS

Ja.e

Ja[mn]e

Ja[^mn]e

ab*c

ab+c

^abc

abc\$

a(bc|de)f

MATCHED STRINGS

“Jame” or “Jane” or “Ja8e”, *not* “Jaco”

“Jame” or “Jane” *only*

“Jake” or “Ja8e” but *not* “Jame” or “Jane”

a[0 or more b]c, e.g. ac, abc, abbc, abbbc

a[1 or more b]c, e.g. abc, abbc, *not* ac

abc *only at the beginning of a line*

abc *only at the end of a line*

abcf *or* adef

Extracting columns: cut

```
cut -c1-5 phonebook
```

```
Jacob  
James  
Jane  
Keith  
Kenny  
Larry
```

```
cut -c1-6,16- phonebook
```

```
Jacob 1234  
James 4567  
Jane 2365  
Keith 5673  
Kenny 0989  
Larry 6575
```

Extracting columns: cut

```
cat temp1
```

```
4319837 0.50000 qpostPro jksmith      Eqw  05/21/2010 14:14:05
4319839 0.50000 qpostPro jksmith      Eqw  05/21/2010 14:14:11
4319841 0.50000 qpostPro jksmith      Eqw  05/21/2010 14:14:17
4319843 0.50000 qpostPro jksmith      Eqw  05/21/2010 14:14:23
```

```
cut -c1-8 temp1
```

```
4319837
4319839
4319841
4319843
```

To avoid counting characters, try 'awk'.

Stream editor sed



sed 's/902/ /' phonebook

substitution

```
Jacob      555 1234
James      555 4567
Jane       555 2365
Keith      555 5673
Kenny      555 0989
Larry      555 6575
```

sed '4,5d' phonebook

deletion

```
Jacob      902 555 1234
James      902 555 4567
Jane       902 555 2365
Larry      902 555 6575
```

sed 's/^/hello /' phonebook

sub at start of line

```
hello Jacob 902 555 1234
hello James 902 555 4567
hello Jane  902 555 2365
hello Keith 902 555 5673
hello Kenny 902 555 0989
hello Larry 902 555 6575
```

Back to the example...

Suppose you have fifty jobs on the cluster, some in *r* state, some in *qw* state and some in *Eqw* state

How do you delete your *Eqw* jobs efficiently?

```
qstat > temp1
grep 'Eqw' temp1 > temp2
cut -c1-8 temp2 > temp3
sed 's/^/qdel /' temp3 > del.sh
source del.sh
rm temp1 temp2 temp3 del.sh
```

Eqw.sh

Use the pipe!

Avoid all those annoying intermediate files with “the pipe”:



“Pipes” output of first command into input of second

pipeEqw.sh

```
qstat | grep Eqw | cut -c1-8 | sed 's/^/qdel /' > del.sh  
source del.sh  
rm del.sh
```

awk

A meaningless name for a pattern scanning *and processing* tool. More powerful than **grep**, but not as powerful as **perl**.

I won't teach you **awk** today. The point is, there's always another way to do it.

```
joblist=`qstat | awk '/ Eqw / {print $1}'`  
qdel $joblist
```


Programming:

Arguments and variables

```
#!/bin/bash
user="$1"
if who | grep "^$user " > /dev/null
then
    echo "$user is logged ON"
else
    echo "$user is not logged on"
fi
```

iftest.sh

- \$1, \$2, *etc.* are script arguments
- \$user is a variable
 - set with “name=”
 - referenced with “\$name”

Programming: if – then – else

If
then
else
fi

```
#!/bin/bash
user="$1"
if who | grep "^$user " > /dev/null
then
    echo "$user is logged ON"
else
    echo "$user is not logged on"
fi
```

iftest.sh

- Obeys *return code* of conditional, not *output*!
 - Experiment with 'echo \$?' to see return codes
 - 0 good → “then” clause; nonzero bad → “else” clause

Programming: if – then – else

If
then
else
fi

```
#!/bin/bash
user="$1"
if who | grep "^$user " > /dev/null
then
    echo "$user is logged ON"
else
    echo "$user is not logged on"
fi
```

iftest.sh

- Dialect-dependent!
 - /bin/tcsh has different syntax
 - first line `#!` tells system which shell to use

File testing example

```
#!/bin/bash

if [ -f $1 ]
then
    echo "Yes, $1 is here"
else
    echo "No file named $1 here"
fi
```

iftest2.sh

Checking files is very common,
so a collection of file tests are supplied...

File tests

Operator	Returns TRUE (exit status of 0) if
<i>-e file</i>	<i>-----> file exists</i>
<i>-d file</i>	<i>-----> file is a directory</i>
<i>-f file</i>	<i>-----> file is an ordinary file</i>
<i>-r file</i>	<i>-----> file is readable by the process</i>
<i>-w file</i>	<i>-----> file is writable by the process</i>
<i>-x file</i>	<i>-----> file is executable</i>
<i>-s file</i>	<i>-----> file has nonzero length</i>
<i>-L file</i>	<i>-----> file is a symbolic link</i>

Another practical example

You want to submit a number of jobs, differing only in the Temperature in the input file and you want each to have a unique job name.

Another practical example



We'll want these files...

run.exe

template.in

sub.sh



```
#####  
#Temperature  
T=1  
#Pressure  
P=1  
#
```



```
#$ -cwd  
#$ -N CHANGE  
#$ -o t.out  
#$ -e t.err  
#$ -l h_rt=12:00:00  
  
./run.exe input"$1".in
```

Programming: 'for' loop



make_input.sh

```
#!/bin/bash
# Make input files for a range of temperatures
j=0
for t in 178 189 208 213 218 228 248
do
    j=$((j+1))
    sed 's/T=1/T='$t'/'      template.in > input"$j".in
    sed 's/CHANGE/job'$j'/' sub.sh      > sub"$j".sh
    # 'echo' what we'd do instead of really submitting
    # the jobs, since this is just a demo:
    echo temperature=$t : qsub sub"$j".sh $j
done
```

Look into “here documents” for a way to do this without 'sed'.
Bash syntax shown, tcsh syntax is different.

Where next?

- Online manual, 'man'
 - `man bash` or `man tcsh`
- Tutorials on the Web
 - Google 'unix shell script tutorial'
- Books
 - *The Unix Programming Environment* by Kernighan & Pike is the classic
 - ...but there are many others!
- ACEnet Wiki
 - <http://www.ace-net.ca/>
- ACEnet Tech Support email
 - `support@ace-net.ca`

Example files

```
brasdor.ace-net.ca:/home/rdickson/demo/Scripting/*  
courtenay.ace-net.ca:/home/rdickson/demo/Scripting/*  
fundy.ace-net.ca:/home/rdickson/demo/Scripting/*  
glooscap.ace-net.ca:/home/rdickson/demo/Scripting/*  
mahone.ace-net.ca:/home/rdickson/demo/Scripting/*  
placentia.ace-net.ca:/home/rdickson/demo/Scripting/*  
  
cp /home/rdickson/demo/Scripting/* .
```